

Part A - Theory

Q1: **Sequential Data:** Sequential data is data where the order of elements is important.

Key characteristics: The core characteristic is that the data points are not independent. They arrive in an ordered sequence and this order is critical to their meaning. A model processes the sequence one step at a time, using an "internal state" to remember past information as it processes new information.

•) **Real world examples:**

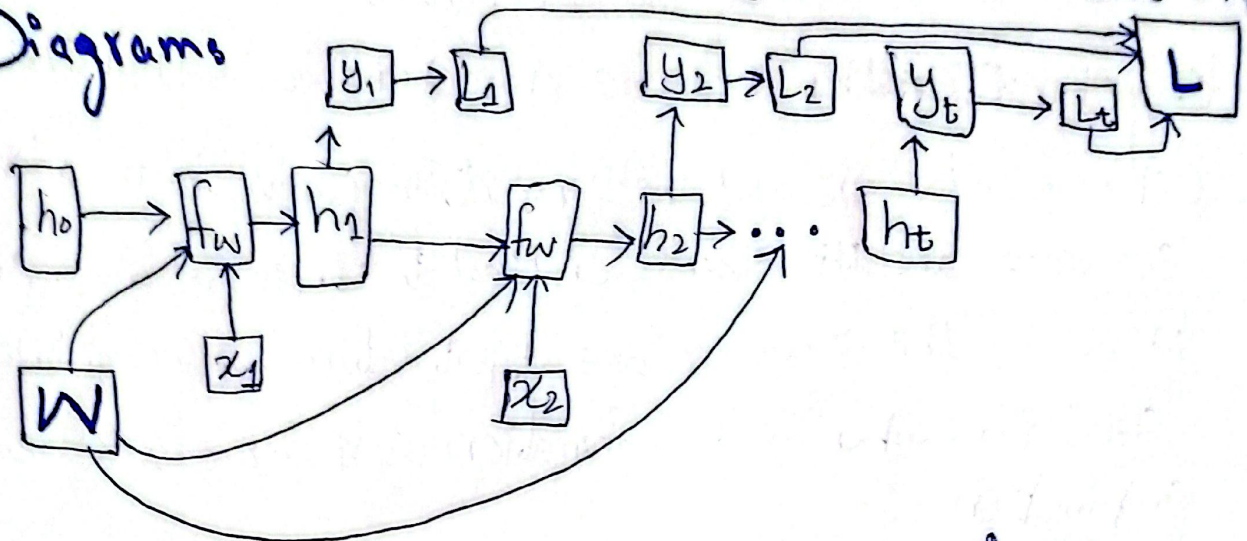
-) Image Captioning (One-to-Many).
-) Action Prediction (Many-to-One).
-) Video Captioning (Many-to-Many).
-) Language Modeling (Many-to-Many).
-) Machine Translation (Many-to-Many).

Q2:- **Architecture of RNNs**

- 1) At each timestep t , the RNN cell receives two inputs: the current vector x_t and the hidden state from previous time step h_{t-1} .
- 2) It computes a new hidden state h_t , using a recurrence formula.

- 3) Critically, the same function and the same set of Parameters (weights) W are used at every single time step.
- 4) The cell can also produce output y_t , at each timestep, which is calculated based on the new hidden state h_t .

Diagrams



($\because$ Where x_i 's are inputs, h_i 's are hidden states, y_i 's are outputs and L_i 's are loss at each time step.
 ($\because$ h_i 's and y_i 's are same).)

Q3:- LSTM Gates:-

The long short term memory (LSTM) network is an advanced RNN variant. Its cell structure is designed to be better at preserving information over long time steps.

structures Unlike a Vanilla RNN, an LSTM cell maintains two states that are passed to the next time step:

- **Cell state (C_t):** This is the long term memory. It's designed so that information can flow along it with only minor linear modifications, making it easy to preserve information.
- **Hidden State (h_t):** This is the short-term memory and

the output for the current time step. It is used to compute the gates for the next time step.

Role of Gates: The gates are the control mechanism for the cell's memory. They are a set of four components computed from the previous hidden state (h_{t-1}) and the current input (x_t). They use sigmoid activation functions to act like "switches" that control flow of information.

- 1) **Forget Gate (f):** A sigmoid gate that decides what information to erase from the old cell state (C_{t-1}).
- 2) **Input Gate (i):** A sigmoid gate that decides if new information should be written to the cell state.
- 3) **Gate Gate (g):** A tanh gate that computes the new candidate information to be added.
- 4) **Output Gate (o):** A sigmoid gate that decides how much of the updated cell state (C_t) to reveal as the new hidden state (h_t).

Q4:- Role of Activations:

Activation functions introduce non-linearity, but their roles are different and more strategic in LSTMs.

- 1) **In Vanilla RNNs:-** A single activation function, typically tanh, is used to compute the new hidden state $h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$. It squashes the output to

be between -1 and 1 .

• **Learning Dynamics:** This non-linearity is a primary cause of vanishing/exploding gradient problem. During backpropagation, the gradient is repeatedly multiplied by the derivative of $\tanh$ and the recurrent weights. This makes it very difficult for the ~~ne~~ network to learn dependencies over long sequences.

2) **In LSTMs:-** LSTMs use two activation functions for different purposes:

• **Sigmoid:** This is used for the gates. Because it outputs a value between 0 and 1 , it acts as a "controller", deciding how much of the information to let pass through each gate.

• **Tanh:** This is used to compute the new candidate values (the "gate gate" g) and to scale the final hidden state output (h_t). It squashes values to be between -1 and 1 , which is suitable for representing the content of the memory.

• **Learning Dynamics:** The element creates an "uninterrupted gradient flow" along the cell state (c_t). The gradient path from c_t to c_{t-1} does not involve a matrix multiplication with the main weights. It's just an element-wise multiplication by the forget gate f . This makes it much easier for the model to learn long-distance dependencies compared to a vanilla RNN.

Q 5:-

Feature

Vanilla RNN

LSTM

Memory Handling

Uses a single hidden state (h_t) which acts as its only memory. This memory is completely recalculated and overwritten at each timestep.

Uses two states: a cell state (c_t) for long-term memory and a hidden state (h_t) for short-term memory. Gates explicitly control what is forgotten, added or read from the cell state.

Gradient Behavior

Suffers from vanishing and exploding gradients. Gradients must flow through the tanh activation and recurrent weight matrix (W_{hh}) at each step.

Mitigates vanishing/exploding gradients. The cell state (c_t) provides an "uninterrupted gradient flow" that is only multiplied element-wise by the forget gate, not by a shared weight matrix.

Performance on long Sequences

Poor. It is difficult to access information from many steps back. Because the memory and gradients fade or explode over time.

Much better. The architecture makes it easier for the RNN to preserve information over many timesteps. The forget gate can learn to hold onto information indefinitely (by setting $f=1$).

Q 6:-

Encoder: The encoder's job is to read the input sequence $(x_1, x_2, \dots, x_n)$ and ~~encode~~ compress it into a fixed-size

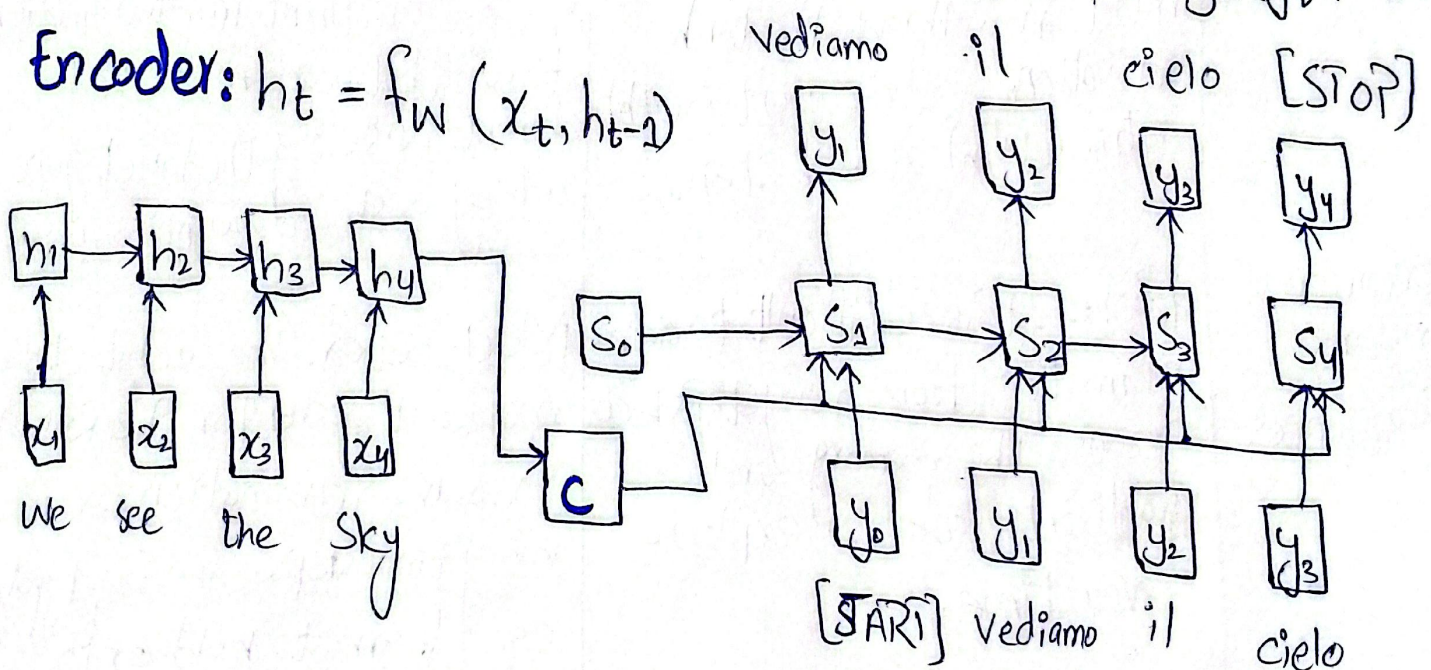
vector. It processes the input one by one, updating its hidden state at each step. Its final hidden state h_n becomes the "Context vector" c which represents a summary of the entire input sequence.

Decoder: The decoder's job is to generate the output sequence $(y_1, y_2, \dots, y_T)$ from this context vector. It is initialized with the context vector c . It then generates the output token by token, using the previously generated output as the input for the next time step.

Diagram:-

$$\text{Decoder: } S_t = g(y_{t-1}, S_{t-1}, c)$$

$$\text{Encoder: } h_t = f_w(x_t, h_{t-1})$$



Q7: Encoder - Decoder Architecture:

An ~~R~~ Encoder reads the entire input sequence $(x_1 \dots x_T)$ and compress it into a single context vector c .

A decoder is initialized with this context vector c and generates

the output sequence one token at a time.

A major limitation of this model is the bottleneck of compressing the entire input into one fixed-size context vector c , which is especially problematic for long sequences.

Translation and Summarization:

- 1) It decouples input and output lengths. The encoder reads any length of input and the decoder produces any length of output.
- 2) The context vector c acts as a bridge or the "thought", that connects the two sequences. The encoder "understands" the source sentence and summarizes it into c , and the decoder "expresses" that summary c in the target language.

Q8:- Part-B - Numericals

Mathematical Expression for Basic RNN:-

- 1) Hidden State (h_t): The new hidden state is calculated by combining the previous hidden state (h_{t-1}) and the current input (x_t), then applying the tanh activation function.

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$

- 2) Output (y_t): The output at the current time step is a linear transformation of the new hidden state.

$$y_t = W_{hy} h_t$$

Q9:- Gating Equations:

•) Input Gate (i_t):

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

•) Forget Gate (f_t):

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

•) "Gate" Gate/candidate (g_t):

$$g_t = \tanh(W_g \cdot [h_{t-1}, x_t] + b_g)$$

•) Output Gate (O_t):

$$O_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

•) Cell state Update: The new cell state is computed by first forgetting old information and then adding new candidate information.

$$C_t = (f_t \odot C_{t-1}) + (i_t \odot g_t)$$

•) Hidden state update: The new hidden state is filtered version of the new cell state, passed through $\tanh$ and controlled by the output gate O_t .

$$h_t = O_t \odot \tanh(C_t)$$

•————•